

Routing.yml: As the name suggests, in this file you specify URLs and the corresponding resources to which Symfony will route the requesting user. Each application has its own routing file in the **Project > Source Files > Apps > [Application Name] > Config** folder. Here is a sample of a file entry:

```
# default rules
homepage:
  url: /
  param: { module: default, action: index }
```

Here, *homepage* is the name of the rule. This says that for the URL '/' (root), redirect to the *index* action of the *default* module. You can add similar entries as your application develops. Note that Symfony searches from the top down, and redirects to the first match found—so you have to add entries accordingly.

Now, read on to learn more about how to get started as an amateur Symfony developer.

Application design

The best way to get started is to let Symfony create an application for you, and then modify it according to your needs. This avoids developing code from scratch—and since you have working code, you will learn the ways of Symfony even faster.

To create an application, first create a Symfony project, set up a database for the application (and don't forget to configure *'databases.yml'* first). Once the database is created, you have to run the *'symfony doctrine:build-schema'* command, which reads the database you have created, and builds a *'schema.yml'* file for it. Due to abstraction via ORM and the database abstraction layer, *'schema.yml'* lets Symfony know about the database tables and fields. (You can also go the other route—create the schema file, and use it to generate the database.)

Now that the schema file is ready, we can proceed to create the application. Just run the *'symfony doctrine:build-model'* command to generate the model files corresponding to each table in the database. After models, the forms have to be created, so run *'symfony doctrine:build-forms'*. Next, create a module with *'symfony doctrine:generate-module'*. Finally, update the *'routing.yml'* file by adding a URL and the corresponding resource name in the file. This completes the process.

Now you can access the application via a Web browser, but don't forget to start your Web server and place the files in the root folder before trying to access them. (For more details on the Web server configuration, refer to the previous article in this series.)

Tips

Once you have a working project with code auto-generated by Symfony, it has to be modified. Before that, here is some information on naming conventions that will help you a lot with coding in Symfony:

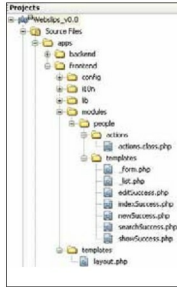


Figure 1: A Symfony project

- Entry in the routing.yml file = [Action Name]
- Function name = *execute[Action Name]*
- Template filename = [Action Name]Success.php

Suppose you want to add a *'Search'* action to your module: the function name added to *'actions.class.php'* should be *'executeSearch'*, and the template file (the view for this action) should be named *'searchSuccess.php'*. The routing file entry

should look like what's shown below:

```
search:
  url: /[Module Name]/search
  param: { module: [Module Name], action: search }
```

Once the action is executed, the output is redirected to the relevant template file (based on the above naming convention), and then the contents of this template file are updated as a PHP variable (*\$sf_content*), which is included in *'layout.php'*, which renders the final output.

Remember that the action file is located in the folder **Project > Source Files > Apps > [App Name] > Modules > [Module Name] > Actions** and the templates are placed in the folder, **Project > Source Files > Apps > [App Name] > Modules > [Module Name] > Templates**.

References and an after-word

You can also refer to the famous Jobee tutorial for Symfony development. Jobee is a demo job-search portal developed on Symfony by Sensio Labs (the people behind Symfony) and is at <http://goo.gl/TltQG>.

With this information, you can now go ahead and develop your application easily with Symfony. Hope you liked the article; for any queries or information, feel free to drop me a mail. Happy coding! 

By: Nitish Tiwari

The author is a FOSS enthusiast, currently working in a FOSS start-up. He is fond of bikes, travelling, Bollywood music, and of course, computers. He also runs www.opensource-blogs.com, a blog on open source tools. You can mail him at nitish.tiwari@technocube.in.